

УЧИЛИЩНА СИСТЕМА

Задача: Реализирайте следната училищна система:

```
D:\Exercises\11 class\List_Class_SchoolSystem\bin\Deb
=== УЧИЛИЩНА СИСТЕМА ===
а) Покажи всички ученици
б) Добави нов ученик
в) Намери най-добрия ученик
г) Среден успех по класове
д) Изход
-----
Изберете опция:
w
Най-добър ученик: Ана с оценка 6.00
-----
Изберете опция:
г
=== СРЕДЕН УСПЕХ ПО КЛАСОВЕ ===
Клас 11 а: 5.62
Клас 11 б: 5.78
Клас 11 в: 4.88
-----
Изберете опция:
б
Име: Петко
Възраст: 17
Оценка: 5.55
Клас: 11 г
Ученикът е добавен успешно!
-----
Изберете опция:
г
```

Възможно решение:

1) Първо правим класа:

```
public class Student
{
    4 references
    public string Name { get; set; }
    2 references
    public int Age { get; set; }
    5 references
    public double Grade { get; set; }
    4 references
    public string Class { get; set; }

    5 references
    public Student(string name, int age, double grade, string studentClass)
    {
        Name = name;
        Age = age;
        Grade = grade;
        Class = studentClass;
    }

    1 reference
    public void DisplayInfo()
    {
        Console.WriteLine($"{Name} ({Class}), {Age}г., оценка: {Grade:F2}");
    }
}
```

2) Менюто, което да се покаже:

```
private static void ShowMenu()
{
    Console.WriteLine(@"=== УЧИЛИЩНА СИСТЕМА ===
а) Покажи всички ученици
б) Добави нов ученик
в) Намери най-добрия ученик
г) Среден успех по класове
г) Изход");
}
```

3) Създаване на нов списък:

0 references

```
static void Main()
{
    List<Student> students = EnterStudents();

    ShowMenu();

    ChooseOptions(students);
}
```

1 reference

```
private static List<Student> EnterStudents()
{
    List<Student> students = new List<Student>();

    students.Add(new Student("Иван", 17, 5.25, "11 а"));
    students.Add(new Student("Мария", 16, 5.75, "11 б"));
    students.Add(new Student("Петър", 17, 4.50, "11 в"));
    students.Add(new Student("Ана", 16, 6.00, "11 а"));
    students.Add(new Student("Димитър", 17, 5.80, "11 б"));
    students.Add(new Student("Желязко", 17, 5.25, "11 в"));

    return students;
}
```

4) Менюто за избор:

```
private static void ChooseOptions(List<Student> students)
{
    while (true)
    {
        Console.WriteLine(new string('-', 37));
        Console.WriteLine("Изберете опция: ");

        char choice = char.Parse(Console.ReadLine().ToLower());

        switch (choice)
        {
            case 'a' or 'A':
                ShowAllStudents(students);
                break;
            case 'b' or 'B':
                AddNewStudent(students);
                break;
            case 'c' or 'C' or 'v' or 'V':
                FindBestStudent(students);
                break;
            case 'd' or 'D':
                ShowClassAverages(students);
                break;
            case 'g' or 'G':
                Console.WriteLine("Довиждане :)");
                return;
            default:
                Console.WriteLine("Невалиден избор!");
                break;
        }
    }
}
```

5) Отделните методи:

- ```
static void ShowAllStudents(List<Student> students)
{
 Console.WriteLine("\n=== ВСИЧКИ УЧЕНИЦИ ===");
 foreach (Student student in students.OrderBy(s => s.Class).ThenBy(s => s.Name))
 {
 student.DisplayInfo();
 }
}
```

`Student student in students.OrderBy(s => s.Class).ThenBy(s => s.Name)`

-

```
static void AddNewStudent(List<Student> students)
{
 Console.Write("Име: ");
 string name = Console.ReadLine();
 Console.Write("Възраст: ");
 int age = int.Parse(Console.ReadLine());
 Console.Write("Оценка: ");
 double grade = Math.Round(double.Parse(Console.ReadLine()), 2);
 Console.Write("Клас: ");
 string studentClass = Console.ReadLine();

 students.Add(new Student(name, age, grade, studentClass));
 Console.WriteLine("Ученикът е добавен успешно!");
}
```

\*

```
static void FindBestStudent(List<Student> students)
{
 if (students.Count == 0)
 {
 Console.WriteLine("Няма ученици в системата!");
 return;
 }

 Student best = students.MaxBy(s => s.Grade);
 Console.WriteLine($"Най-добър ученик: {best.Name} с оценка {best.Grade:F2}");
}
```

\*

```
static void ShowClassAverages(List<Student> students)
{
 var classAverages = students
 .GroupBy(s => s.Class)
 .Select(g => new { Class = g.Key, Average = g.Average(s => s.Grade) });

 Console.WriteLine("\n=== СРЕДЕН УСПЕХ ПО КЛАСОВЕ ===");
 foreach (var ca in classAverages)
 {
 Console.WriteLine($"Клас {ca.Class}: {ca.Average:F2}");
 }
}
```

Най-пипкавият момент: да се намери средния успех по класове

```
var classAverages = students
 .GroupBy(s => s.Class)
 .Select(g => new { Class = g.Key, Average = g.Average(s => s.Grade) });

Console.WriteLine("\n=== СРЕДЕН УСПЕХ ПО КЛАСОВЕ ===");
foreach (var ca in classAverages)
{
 Console.WriteLine($"Клас {ca.Class}: {ca.Average:F2}");
}
```

### Целият код:

```
namespace List_Class_SchoolSystem
{
 using System;
 using System.Collections.Generic;
 using System.Linq;

 class Program
 {
 static void Main()
 {
 List<Student> students = EnterStudents();

 ShowMenu();

 ChooseOptions(students);
 }

 private static List<Student> EnterStudents()
 {
 List<Student> students = new List<Student>();

 students.Add(new Student("Иван", 17, 5.25, "11 а"));
 students.Add(new Student("Мария", 16, 5.75, "11 б"));
 students.Add(new Student("Петър", 17, 4.50, "11 в"));
 students.Add(new Student("Ана", 16, 6.00, "11 а"));
 students.Add(new Student("Димитър", 17, 5.80, "11 б"));
 students.Add(new Student("Желязко", 17, 5.25, "11 в"));

 return students;
 }

 private static void ChooseOptions(List<Student> students)
 {
 while (true)
 {
 Console.WriteLine(new string('-', 37));
 Console.WriteLine("Изберете опция: ");

 char choice = char.Parse(Console.ReadLine().ToLower());

 switch (choice)
 {
 case 'a' or 'A':
 ShowAllStudents(students);
 break;
 case 'b' or 'B':
 AddNewStudent(students);
 break;
 case 'v' or 'V' or 'w':
 FindBestStudent(students);
 break;
 case 'r' or 'R':
 ShowClassAverages(students);
 break;
 case 'd' or 'D':
 Console.WriteLine("Довиждане :)");
 break;
 }
 }
 }
 }
}
```

```

 return;
 default:
 Console.WriteLine("Невалиден избор!");
 break;
 }
}

private static void ShowMenu()
{
 Console.WriteLine(@"=== УЧИЛИЩНА СИСТЕМА ===
а) Покажи всички ученици
б) Добави нов ученик
в) Намери най-добрия ученик
г) Среден успех по класове
д) Изход");
}

static void ShowAllStudents(List<Student> students)
{
 Console.WriteLine("\n=== ВСИЧКИ УЧЕНИЦИ ===");
 foreach (Student student in students.OrderBy(s => s.Class).ThenBy(s =>
s.Name))
 {
 student.DisplayInfo();
 }
}

static void AddNewStudent(List<Student> students)
{
 Console.Write("Име: ");
 string name = Console.ReadLine();
 Console.Write("Възраст: ");
 int age = int.Parse(Console.ReadLine());
 Console.Write("Оценка: ");
 double grade = Math.Round(double.Parse(Console.ReadLine()), 2);
 Console.Write("Клас: ");
 string studentClass = Console.ReadLine();

 students.Add(new Student(name, age, grade, studentClass));
 Console.WriteLine("Ученикът е добавен успешно!");
}

static void FindBestStudent(List<Student> students)
{
 if (students.Count == 0)
 {
 Console.WriteLine("Няма ученици в системата!");
 return;
 }

 Student best = students.MaxBy(s => s.Grade);
 Console.WriteLine($"Най-добър ученик: {best.Name} с оценка
{best.Grade:F2}");
}

static void ShowClassAverages(List<Student> students)
{

```

```

 var classAverages = students
 .GroupBy(s => s.Class)
 .Select(g => new { Class = g.Key, Average = g.Average(s => s.Grade)
});

 Console.WriteLine("\n=== СРЕДЕН УСПЕХ ПО КЛАСОВЕ ===");
 foreach (var ca in classAverages)
 {
 Console.WriteLine($"Клас {ca.Class}: {ca.Average:F2}");
 }
 }

 public class Student
 {
 public string Name { get; set; }
 public int Age { get; set; }
 public double Grade { get; set; }
 public string Class { get; set; }

 public Student(string name, int age, double grade, string studentClass)
 {
 Name = name;
 Age = age;
 Grade = grade;
 Class = studentClass;
 }

 public void DisplayInfo()
 {
 Console.WriteLine($"{Name} ({Class}), {Age}г., оценка: {Grade:F2}");
 }
 }
}

```